UNIVERSIDADE FEDERAL DE LAVRAS
Departamento de Ciência da Computação
Mestrado/Bacharelado em Ciência da Computação
Disciplina: **Arquitetura de Computadores II - GCC123/PCC507**
**Trabalho Prático 1** – 2º Semestre de 2026
Professor: **Luiz Henrique A. Correia**
Data de entrega: 16/10/2026 – Valor: 15 pontos

# 1 Introdução

O EduMIPS64 é um simulador educacional de CPU baseado na arquitetura MIPS64, desenvolvido em Java e distribuído como software livre [1]. O simulador foi concebido para apoiar o ensino de Arquitetura de Computadores, permitindo a observação da execução de instruções, da utilização dos registradores e da memória, bem como do comportamento das instruções nos diferentes estágios do pipeline.

O objetivo deste trabalho prático é consolidar, de forma quantitativa e experimental, os conceitos de **Métricas de Desempenho**, **Arquitetura do Conjunto de Instruções (ISA)**, **Pipeline**, **Hazards**, **Forwarding** e **Paralelismo em Nível de Instrução (ILP)**, abordados nos capítulos 1 a 3 da disciplina [4].

O trabalho será desenvolvido segundo uma abordagem experimental composta pelas seguintes etapas:

1. compreensão da ISA MIPS64 utilizada pelo simulador;
2. construção de pequenos experimentos controlados sobre o pipeline;
3. identificação e classificação de dependências e hazards;
4. formulação de previsões teóricas antes da execução das simulações;
5. execução dos experimentos com diferentes configurações de hardware;
6. implementação de programas em Assembly MIPS64;
7. análise quantitativa das execuções;
8. otimização do código por reorganização de instruções;
9. comparação objetiva entre as versões original e otimizada.

Os grupos deverão desenvolver programas em linguagem de montagem (*Assembly MIPS64*), executar experimentos no EduMIPS64 e produzir um relatório técnico que apresente não apenas os resultados obtidos, mas também uma explicação causal para o comportamento observado no pipeline [2, 3].

# 2 Objetivos de Aprendizagem

Ao final do trabalho, espera-se que o aluno seja capaz de:

- compreender a relação entre ISA e microarquitetura;
- identificar os estágios IF, ID, EX, MEM e WB de um pipeline clássico;
- identificar dependências de dados entre instruções;
- distinguir dependência de dados de hazard de pipeline;
- reconhecer situações que provocam *data stalls* e *branch stalls*;

- explicar o funcionamento e as limitações do mecanismo de *forwarding*;
- calcular e interpretar CPI, número de ciclos e *speedup*;
- analisar o impacto de desvios condicionais no desempenho;
- aplicar técnicas simples de *instruction scheduling*;
- otimizar uma sequência de instruções sem alterar sua semântica;
- relacionar resultados experimentais aos conceitos teóricos estudados na disciplina.

# 3 Requisitos do Relatório Técnico

O grupo deverá entregar um relatório em formato PDF contendo obrigatoriamente as seções apresentadas a seguir.

## 3.1 Introdução e Construção do Simulador

**O que deve conter:**

Uma breve contextualização sobre a importância de simuladores no projeto, ensino e análise de arquiteturas de computadores.

**Pesquisa acadêmica:**

O grupo deverá documentar e explicar brevemente:

- o objetivo pedagógico do EduMIPS64;
- o histórico do projeto;
- as principais tecnologias utilizadas em sua implementação;
- as diferenças gerais entre as versões Desktop e Web;
- a proposta de visualização da execução baseada nos estágios do pipeline.

Não é necessário realizar uma descrição extensa do código-fonte interno do simulador. O objetivo desta seção é contextualizar a ferramenta utilizada nos experimentos.

## 3.2 Descrição da ISA (*Instruction Set Architecture*)

O grupo deverá apresentar uma descrição técnica do subconjunto da ISA MIPS64 utilizado nos programas desenvolvidos.

Devem ser discutidos, no mínimo:

- quantidade e tamanho dos registradores;
- tamanho das instruções;
- características gerais da filosofia RISC;
- formatos de instrução relevantes;
- instruções aritméticas e lógicas utilizadas;
- instruções de acesso à memória;
- instruções de controle de fluxo;
- registradores utilizados pelos programas desenvolvidos.

Para cada programa, não é necessário descrever toda a ISA MIPS64, mas o grupo deverá explicar adequadamente as instruções efetivamente utilizadas.

### 3.3 Metodologia Experimental

Os experimentos deverão ser planejados antes da execução no simulador.

Para cada experimento, o grupo deverá documentar:

1. objetivo do experimento;
2. trecho de código analisado;
3. dependências esperadas;
4. hazards esperados;
5. comportamento previsto do pipeline;
6. configuração utilizada no EduMIPS64;
7. resultado observado;
8. explicação para eventuais diferenças entre previsão e resultado.

Todas as execuções deverão considerar, quando aplicável, os seguintes cenários:

- **Cenário A – Forwarding OFF:** pipeline com o mecanismo de adiantamento (*Forwarding*) desativado.
- **Cenário B – Forwarding ON:** pipeline com o mecanismo de adiantamento (*Forwarding*) ativado.

**Observação:** a configuração de *Forwarding* encontra-se nas configurações gerais do simulador, associada às opções de Pipeline.

Sempre que possível, todas as demais configurações deverão permanecer constantes entre duas execuções comparadas.

## 4 Parte I – Microexperimentos Obrigatórios

Antes da implementação dos programas principais, o grupo deverá realizar os seguintes microexperimentos.

O objetivo desta etapa é isolar fenômenos específicos do pipeline antes de observá-los em programas maiores.

### 4.1 Experimento 1 – Pipeline sem Dependências

Construir uma sequência contendo pelo menos quatro instruções independentes.

Exemplo conceitual:

```
DADDI R1, R0, 10
DADDI R2, R0, 20
DADDI R3, R0, 30
DADDI R4, R0, 40
```

O grupo deverá:

- representar graficamente a ocupação do pipeline;
- identificar os ciclos de entrada e saída de cada instrução;
- calcular o número total de ciclos;
- calcular o CPI;
- explicar por que esse cenário se aproxima do comportamento ideal do pipeline.

## 4.2 Experimento 2 – Dependências RAW e Forwarding

Construir uma sequência contendo uma cadeia explícita de dependências do tipo RAW (*Read After Write*).

Exemplo conceitual:

```
DADDI R1, R0, 10
DADD  R2, R1, R3
DADD  R4, R2, R5
DADD  R6, R4, R7
```

Antes da simulação, o grupo deverá identificar as dependências na forma:

| Instrução produtora | Instrução consumidora | Registrador |
|---|---|---|
| I1 | I2 | R1 |
| I2 | I3 | R2 |
| I3 | I4 | R4 |

Em seguida, executar a mesma sequência com:

- Forwarding OFF;
- Forwarding ON.

O grupo deverá explicar quais stalls foram removidos ou reduzidos e por quê.

## 4.3 Experimento 3 – Dependência após Carregamento

Construir uma sequência contendo uma instrução de leitura da memória imediatamente seguida por uma instrução dependente do valor carregado.

Exemplo conceitual:

```
LD    R1, 0(R2)
DADD  R3, R1, R4
```

O grupo deverá investigar experimentalmente:

- se o Forwarding elimina completamente a penalidade;
- em qual estágio o dado necessário se torna disponível;
- em qual estágio a instrução seguinte necessita desse dado;
- se ocorre a inserção de alguma bolha no pipeline.

Em seguida, inserir uma instrução independente entre as duas operações:

```
LD    R1, 0(R2)
DADD  R7, R8, R9
DADD  R3, R1, R4
```

Comparar quantitativamente as duas execuções e explicar o efeito da reorganização.

### 4.4 Experimento 4 – Hazard de Controle

Construir um pequeno laço contendo um desvio condicional.

Exemplo conceitual:

```
DADDI R1, R0, 10
DADDI R2, R0, 0

loop:
DADDI R2, R2, 1
DADDI R1, R1, -1
BNE   R1, R0, loop
```

O grupo deverá analisar:

- quantidade de desvios executados;
- quantidade de *branch stalls*;
- impacto dos desvios no número total de ciclos;
- impacto dos desvios no CPI;
- proporção dos ciclos associada a penalidades de controle.

## 5 Parte II – Programas Desenvolvidos

O grupo deverá desenvolver **3 (três) programas inéditos** em Assembly MIPS64.

Os programas deverão ser diferentes dos microexperimentos apresentados anteriormente.

### 5.1 Programa 1 – Operações sobre Vetores

Implementar um programa que realize uma operação não trivial sobre um vetor.

Exemplos:

- soma de dois vetores;
- produto escalar;
- cálculo do maior ou menor elemento;
- soma acumulada;
- transformação elemento a elemento.

O programa deverá envolver:

- acesso à memória;
- operações aritméticas;
- repetição por meio de um laço;
- pelo menos uma dependência RAW;
- pelo menos um desvio condicional.

## 5.2 Programa 2 – Busca, Ordenação ou Processamento

Implementar um algoritmo que envolva decisões e repetição.

Exemplos:

- busca linear;
- ordenação;
- contagem de ocorrências;
- determinação de máximo ou mínimo;
- processamento de uma sequência numérica.

O programa deverá apresentar situações que permitam analisar hazards de dados e de controle.

## 5.3 Programa 3 – Kernel Computacional

Implementar um pequeno kernel computacional de maior complexidade.

Sugestões:

- multiplicação ou operação com matrizes;
- processamento de vetores;
- cálculo numérico iterativo;
- algoritmo de agregação;
- outro problema aprovado pelo professor.

Este será o programa utilizado obrigatoriamente na etapa de otimização descrita posteriormente.

## 5.4 Requisitos Gerais do Código

Todos os códigos deverão:

- executar corretamente no EduMIPS64;
- apresentar entradas e resultados verificáveis;
- conter comentários suficientes para explicar a finalidade das instruções;
- utilizar registradores de forma coerente;
- conter dependências de dados relevantes para análise;
- utilizar pelo menos uma estrutura de controle baseada em desvio;
- apresentar organização e legibilidade adequadas.

Não será avaliada positivamente a criação artificial e desnecessária de hazards apenas para aumentar sua quantidade. Os hazards analisados deverão estar associados ao funcionamento natural ou a uma versão experimental claramente identificada do programa.

# 6 Validação Funcional

Antes da análise de desempenho, o grupo deverá demonstrar que cada programa produz resultados corretos.

Para cada programa deverá ser apresentada pelo menos uma entrada de teste e sua respectiva saída esperada.

Exemplo:

| Entrada | Saída esperada |
|---|---|
| $[5, 2, 8, 1]$ | $[1, 2, 5, 8]$ |

A análise de desempenho somente será considerada válida para programas funcionalmente corretos.

# 7 Previsão Antes da Simulação

Antes de executar cada programa principal, o grupo deverá selecionar um trecho crítico e apresentar uma previsão do seu comportamento.

Deverão ser indicados:

- instruções produtoras e consumidoras;
- registradores envolvidos;
- dependências RAW identificadas;
- possíveis stalls;
- efeito esperado do Forwarding;
- impacto esperado de instruções de desvio.

Sempre que possível, deverá ser apresentada uma tabela contendo valores previstos e observados.

Exemplo:

| Métrica | Previsto | Observado | Diferença |
|---|---|---|---|
| Ciclos | – | – | – |
| Data stalls | – | – | – |
| Branch stalls | – | – | – |
| CPI | – | – | – |

Diferenças entre previsão e resultado observado deverão ser explicadas.

# 8 Análise Detalhada do Pipeline

O grupo deverá selecionar um trecho crítico contendo entre **4 e 8 instruções consecutivas** de um dos programas.

A execução deverá ser acompanhada ciclo a ciclo nos cinco estágios:

- IF – Instruction Fetch;
- ID – Instruction Decode;
- EX – Execute;

- MEM – Memory Access;
- WB – Write Back.

A análise deverá ser apresentada, preferencialmente, na forma de uma tabela semelhante à seguinte:

| Ciclo | I1 | I2 | I3 | I4 | Evento |
|---|---|---|---|---|---|
| 1 | IF | | | | |
| 2 | ID | IF | | | |
| 3 | EX | ID | IF | | |
| 4 | MEM | Stall | ID | IF | Hazard RAW |
| 5 | WB | EX | Stall | ID | Encaminhamento de dado |

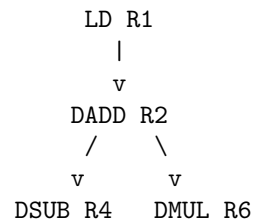
O grupo deverá identificar explicitamente:

- momento em que cada dependência aparece;
- origem da dependência;
- bolhas inseridas;
- utilização do mecanismo de Forwarding;
- consequências sobre o número total de ciclos.

# 9 Grafo de Dependências

Para pelo menos um trecho de um dos programas principais, deverá ser construído um grafo de dependências entre instruções.

Exemplo conceitual:




A partir do grafo, o grupo deverá responder:

- quais instruções possuem dependências verdadeiras;
- quais instruções são independentes entre si;
- onde existe oportunidade de Paralelismo em Nível de Instrução;
- quais instruções poderiam ser reorganizadas sem alterar a semântica do programa.

# 10 Otimização por Reordenação de Instruções

O Programa 3 deverá possuir duas versões:

1. **Versão Original**;
2. **Versão Otimizada**.

A versão otimizada deverá preservar integralmente o resultado computacional da versão original.
O grupo poderá utilizar técnicas como:

- reorganização de instruções independentes;
- preenchimento de intervalos entre produção e consumo de dados;
- redução de dependências desnecessárias;
- melhor utilização de registradores;
- eliminação de operações redundantes;
- reorganização de laços, quando aplicável.

Não será permitido alterar o problema computacional apenas para obter uma redução artificial no número de ciclos.
A comparação deverá apresentar:

| Versão | IC | CC | Data Stalls | Branch Stalls | CPI |
|---|---|---|---|---|---|
| Original | – | – | – | – | – |
| Otimizada | – | – | – | – | – |

O grupo deverá explicar quais transformações foram realizadas e por que elas afetaram o pipeline.

# 11 Análise Quantitativa de Desempenho

Para os experimentos relevantes, deverão ser coletadas as seguintes métricas:

- Contagem de Instruções ($IC$);
- Ciclos de Clock Totais ($CC$);
- Data Stalls;
- Branch Stalls;
- CPI Global;
- Speedup entre configurações ou versões.

O CPI deverá ser calculado por:

$$CPI = \frac{CC}{IC} \tag{1}$$

Quando forem comparadas duas execuções do mesmo programa, mantendo constante o período de clock, o grupo poderá calcular o *speedup* por:

$$Speedup = \frac{CC_{\text{referência}}}{CC_{\text{otimizado}}} \tag{2}$$

Os resultados deverão ser apresentados em tabelas comparativas.
Exemplo:

| Configuração | IC | CC | Data Stalls | Branch Stalls | CPI |
|---|---|---|---|---|---|
| Forwarding OFF | – | – | – | – | – |
| Forwarding ON | – | – | – | – | – |
| Otimizada | – | – | – | – | – |

## 12 Experimento com Diferentes Tamanhos de Entrada

Para pelo menos um dos programas principais, deverão ser utilizadas diferentes dimensões de entrada.

Sugestão:

$$N \in \{5, 10, 20, 50\}$$

ou valores equivalentes adequados ao algoritmo escolhido.

O grupo deverá observar como variam:

- número de instruções;
- ciclos;
- Data Stalls;
- Branch Stalls;
- CPI.

Uma tabela semelhante à seguinte deverá ser apresentada:

| N | IC | CC | Data Stalls | Branch Stalls | CPI |
|---|---|---|---|---|---|
| 5 | – | – | – | – | – |
| 10 | – | – | – | – | – |
| 20 | – | – | – | – | – |
| 50 | – | – | – | – | – |

O grupo deverá discutir se o comportamento observado para pequenas entradas é representativo do comportamento do programa para entradas maiores.

## 13 Discussão dos Resultados

Esta seção deverá ir além da simples apresentação das métricas.

O grupo deverá responder, com base nos experimentos realizados, às seguintes questões:

1. Qual foi o impacto do Forwarding sobre o número total de ciclos?
2. Qual foi o impacto do Forwarding sobre o CPI?
3. Quais dependências foram resolvidas ou tiveram sua penalidade reduzida pelo Forwarding?
4. Existem situações em que ainda ocorrem stalls mesmo com Forwarding habilitado? Explique a causa.
5. Qual foi o impacto dos desvios condicionais no desempenho?
6. Em quais trechos havia oportunidades de execução independente entre instruções?
7. A reorganização do código reduziu o número de stalls?

8. A otimização alterou o número de instruções executadas?
9. Qual foi o speedup observado entre as versões comparadas?
10. Uma redução de CPI necessariamente implica redução proporcional no tempo de execução? Discuta considerando as métricas avaliadas.
11. Quais mudanças apresentaram maior impacto no desempenho e por quê?

As respostas deverão ser fundamentadas nos resultados experimentais obtidos pelo grupo.

# 14 Guia de Reprodução do Experimento

O relatório deverá conter um pequeno guia para que outra pessoa consiga reproduzir os experimentos. Devem ser descritos:

1. versão utilizada do EduMIPS64;
2. utilização da versão Web ou Desktop;
3. procedimento para carregar os arquivos `.s`;
4. configurações de pipeline utilizadas;
5. procedimento para ativar/desativar Forwarding;
6. procedimento para execução passo a passo (*Step*);
7. procedimento para execução completa (*Run*);
8. forma de obtenção das estatísticas utilizadas no relatório;
9. valores de entrada utilizados nos programas.

O objetivo é garantir a **reprodutibilidade** dos resultados apresentados.

# 15 Arquivos a Serem Entregues

A entrega deverá conter:

1. relatório técnico em formato PDF;
2. código-fonte de todos os microexperimentos;
3. código-fonte dos três programas principais;
4. versão original e versão otimizada do Programa 3;
5. arquivo README contendo instruções rápidas de execução;
6. tabelas, imagens ou demais evidências utilizadas na análise.

Sugere-se a seguinte estrutura:

```
grupoXX/
|
+-- README.md
+-- relatorio.pdf
|
+-- microexperimentos/
|   +-- exp1_pipeline_ideal.s
|   +-- exp2_raw.s
|   +-- exp3_load_use.s
|   +-- exp4_branch.s
|
+-- programas/
|   +-- programa1.s
|   +-- programa2.s
|   +-- programa3_original.s
|   +-- programa3_otimizado.s
|
+-- resultados/
    +-- tabelas/
    +-- imagens/
```

# 16 Critérios de Avaliação

O trabalho possui valor total de **15 pontos**, distribuídos conforme os seguintes critérios:

| Critério | Valor |
|---|---|
| Descrição da ISA e fundamentação conceitual | 1,5 |
| Microexperimentos e previsões | 2,0 |
| Correção funcional dos programas | 2,0 |
| Análise ciclo a ciclo do pipeline | 2,0 |
| Análise quantitativa de desempenho | 2,5 |
| Otimização e análise de ILP | 2,0 |
| Discussão e interpretação dos resultados | 2,0 |
| Reprodutibilidade, organização e documentação | 1,0 |
| **Total** | **15,0** |

Será valorizada principalmente a capacidade de **explicar os fenômenos observados**, e não apenas de apresentar capturas de tela ou valores produzidos pelo simulador.

Resultados sem interpretação técnica adequada poderão receber pontuação parcial mesmo quando numericamente corretos.

# 17 Informações da Entrega

Os grupos devem ser formados por **EXATAMENTE 4 (quatro) discentes**, com exceção dos alunos do Mestrado, para os quais serão permitidos grupos com no máximo 3 membros.

A formação do grupo deverá ser enviada por e-mail para:

`lcorreia@ufla.br`

até, no máximo, o dia **25/09/2026**.

**Data de entrega: 16/10/2026.**

## Referências Bibliográficas


[1] EDUMIPS64. *EduMIPS64/edumips64: Free cross-platform educational MIPS64 CPU Simulator*. GitHub, 2026. Disponível em: `https://github.com/EduMIPS64/edumips64`. Acesso em: 24 maio 2026.

[2] EDUMIPS64. *EduMIPS64 Home Page*. 2026. Disponível em: `https://edumips.org`. Acesso em: 24 maio 2026.

[3] EDUMIPS64. *EduMIPS64 Experimental Web Frontend*. 2026. Disponível em: `https://web.edumips.org`. Acesso em: 24 maio 2026.

[4] CORREIA, Luiz H. A. *GCC123 – Arquitetura de Computadores II*. Universidade Federal de Lavras, 2026. Material didático da disciplina. Disponível em: Campus Virtual da UFLA. Acesso em: 24 maio 2026.